

Go for Senior .NET Engineers

Phase 2: Type System & Design Patterns

9 Detailed Lessons with C# Comparisons & Production Patterns

Current as of Go 1.25.6 (January 2026)

Covers: Generics, Reflection, Functional Options, DI, and Architectural Patterns

Lesson 16: Interface Composition — Small Interfaces

Go's standard library is built on interfaces with 1-3 methods. This is the opposite of C#'s tendency toward large interfaces like `IRepository<T>` with 10+ methods. Understanding this philosophy is essential for writing idiomatic Go.

16.1 — The `io.Reader` / `io.Writer` Foundation

```
// The two most important interfaces in Go:
type Reader interface {
    Read(p []byte) (n int, err error)
}

type Writer interface {
    Write(p []byte) (n int, err error)
}

// Composition – combine small interfaces into larger ones
type ReadWriter interface {
    Reader
    Writer
}

type ReadWriteCloser interface {
    Reader
    Writer
    Closer
}
```

16.2 — C# `IEnumerable<T>` vs Go Composable Interfaces

C# / .NET

```
// C# – large interface hierarchy
public interface ICollection<T>
    : IEnumerable<T> {
    int Count { get; }
    void Add(T item);
    void Clear();
    bool Contains(T item);
    void CopyTo(T[] arr, int idx);
    bool Remove(T item);
}
// 6 methods + inherited IEnumerable
```

Go

```
// Go – compose tiny interfaces
type Reader interface {
    Read(p []byte) (n int, err error)
}

type Closer interface {
    Close() error
}

type ReadCloser interface {
    Reader
    Closer
}
// Each interface: 1 method. Compose as needed.
```

16.3 — Standard Library Interface Examples

Interface	Methods
io.Reader	Read([]byte) (int, error)
io.Writer	Write([]byte) (int, error)
io.Closer	Close() error
fmt.Stringer	String() string
error	Error() string
sort.Interface	Len(), Less(i,j), Swap(i,j)
http.Handler	ServeHTTP(w, r)
encoding.BinaryMarshaler	MarshalBinary() ([]byte, error)

16.4 — Designing Your Own Composable Interfaces

```
// Step 1: Define tiny, focused interfaces
type UserFinder interface {
    FindByID(ctx context.Context, id string) (*User, error)
}

type UserCreator interface {
    Create(ctx context.Context, u *User) error
}

type UserDeleter interface {
    Delete(ctx context.Context, id string) error
}

// Step 2: Compose when you need more
type UserReadWrite interface {
    UserFinder
    UserCreator
}

// Step 3: Accept the smallest interface you need
func GetUserHandler(finder UserFinder) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        user, err := finder.FindByID(r.Context(), r.PathValue("id"))
        // ... handle response
    }
}
```

TIP: Go proverb: 'Accept interfaces, return structs.' Your functions should accept the smallest interface needed, but return concrete types. This gives callers maximum flexibility.

Lesson 17: Empty Interface `interface{} / any`

The empty interface has zero methods, so every type satisfies it. It is Go's equivalent of C#'s object type. Since Go 1.18, the alias 'any' is preferred over 'interface{}'.

17.1 — any as a Universal Container

```
// Before Go 1.18:
var data interface{} = "hello"

// Go 1.18+ (preferred):
var data any = "hello"
data = 42      // any type works
data = true    // reassigned to bool

// Common usage: generic containers, JSON parsing
var result map[string]any
json.Unmarshal(rawJSON, &result)
```

17.2 — C# object vs Go any

C# / .NET

```
// C# – object boxing
object val = 42;
int n = (int)val; // unbox + cast

// dynamic (runtime dispatch)
dynamic d = GetValue();
d.SomeMethod(); // runtime resolution
```

Go

```
// Go – any (no boxing for interfaces)
var val any = 42
n := val.(int) // type assertion

// No dynamic dispatch equivalent
// Must use type assertion or switch
// to recover the concrete type
```

17.3 — When to Use any

Use any sparingly. It throws away compile-time type safety. Legitimate uses include JSON parsing into unknown structures, plugin systems, and framework-level code. With generics (Go 1.18+), many former uses of `interface{}` are now better served by type parameters.

```
// BEFORE generics – used interface{}
func Contains(slice []interface{}, item interface{}) bool {
    for _, v := range slice {
        if v == item { return true }
    }
    return false
}

// AFTER generics – use type parameter
func Contains[T comparable](slice []T, item T) bool {
    for _, v := range slice {
        if v == item { return true }
    }
    return false
}
```

```
}
```

WARNING: If you find yourself using any everywhere, you're probably fighting the type system. In most cases, generics or a well-defined interface is the better solution.

Lesson 18: Type Assertions & Type Switches

When you have a value of interface type (including any), you need to recover the concrete type. Go provides type assertions and type switches — analogous to C#'s `is/as` patterns and `switch` on type.

18.1 — Type Assertion

```
var val any = "hello"

// Single-value assertion (panics if wrong type)
s := val.(string)
fmt.Println(s) // "hello"

// Safe assertion with comma-ok (never panics)
s, ok := val.(string)
if ok {
    fmt.Println("It is a string:", s)
} else {
    fmt.Println("Not a string")
}

// Assert to interface
var w any = os.Stdout
if writer, ok := w.(io.Writer); ok {
    writer.Write([]byte("works!"))
}
```

18.2 — Type Switch

C# / .NET

```
// C# – pattern matching (C# 7+)
switch (obj) {
    case int n:
        Console.WriteLine($"int: {n}");
        break;
    case string s:
        Console.WriteLine($"str: {s}");
        break;
    case null:
        Console.WriteLine("null");
        break;
    default:
        Console.WriteLine("unknown");
        break;
}
```

Go

```
// Go – type switch
switch v := val.(type) {
case int:
    fmt.Printf("int: %d\n", v)
case string:
    fmt.Printf("str: %s\n", v)
case nil:
    fmt.Println("nil")
default:
    fmt.Printf("unknown: %T\n", v)
}
// Note: v is typed per case branch
// No break needed, no fall-through
```

18.3 — Real-World: Custom Error Handling

```
func HandleError(err error) {
```

```
switch e := err.(type) {  
case *ValidationError:  
    fmt.Printf("Validation: field=%s msg=%s\n", e.Field, e.Message)  
case *NotFoundError:  
    fmt.Printf("Not found: %s\n", e.Resource)  
case *TimeoutError:  
    fmt.Println("Request timed out, retrying...")  
default:  
    fmt.Printf("Unexpected error: %v\n", err)  
}
```

18.4 — Interface Satisfaction Check at Compile Time

```
// Compile-time check: does *MyType satisfy io.Writer?  
var _ io.Writer = (*MyType)(nil)  
  
// If MyType doesn't have Write([]byte)(int,error),  
// this line fails at compile time – no runtime surprise.
```

TIP: Prefer `errors.Is()` and `errors.As()` over type switches for error handling. Type switches on errors break when error wrapping is used.

Lesson 19: Generics (Go 1.18+) — Type Parameters & Constraints

Generics arrived in Go 1.18 (March 2022) and have been refined through Go 1.25. They are deliberately simpler than C# generics — no variance (in/out), no where T : class constraints, and the community ethos is 'use generics sparingly.' Go 1.24 added generic type aliases, and Go 1.25 removed the 'core types' concept for cleaner rules.

19.1 — Basic Generic Function

C# / .NET

```
// C#
T Max<T>(T a, T b)
    where T : IComparable<T> {
    return a.CompareTo(b) > 0
        ? a : b;
}

var m = Max(3, 7); // inferred
```

Go

```
// Go
func Max[T cmp.Ordered](a, b T) T {
    if a > b {
        return a
    }
    return b
}

m := Max(3, 7) // inferred
```

19.2 — Built-in Constraints

Constraint	What It Allows
any	Any type (like C# where T : class removed)
comparable	Types supporting == and != (map keys)
cmp.Ordered	Types supporting < > <= >= (numbers, strings)
~int ~string	Union of underlying types (tilde = includes derived types)

19.3 — Custom Constraints

```
// Define your own constraint as an interface
type Number interface {
    ~int | ~int8 | ~int16 | ~int32 | ~int64 |
    ~uint | ~uint8 | ~uint16 | ~uint32 | ~uint64 |
    ~float32 | ~float64
}

func Sum[T Number](nums []T) T {
    var total T
    for _, n := range nums {
        total += n
    }
}
```



```
    }
    return total
}

// The ~ (tilde) means "any type whose underlying type is..."
type Celsius float64 // underlying type is float64
temps := []Celsius{36.6, 37.2, 38.1}
Sum(temps) // works because ~float64 includes Celsius
```

19.4 — Generic Structs

```
// Generic Stack – like Stack<T> in C#
type Stack[T any] struct {
    items []T
}

func NewStack[T any]() *Stack[T] {
    return &Stack[T]{items: make([]T, 0)}
}

func (s *Stack[T]) Push(item T) {
    s.items = append(s.items, item)
}

func (s *Stack[T]) Pop() (T, bool) {
    if len(s.items) == 0 {
        var zero T
        return zero, false
    }
    item := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return item, true
}

// Usage
intStack := NewStack[int]()
intStack.Push(42)
val, ok := intStack.Pop() // 42, true
```

19.5 — Generic Type Aliases (Go 1.24+)

```
// Go 1.24 introduced generic type aliases
type Set[T comparable] = map[T]bool

mySet := Set[string]{"go": true, "rust": true}
fmt.Println(mySet["go"]) // true
fmt.Printf("%T\n", mySet) // map[string]bool
```

19.6 — Utility Functions: Map, Filter, Reduce

```
func Map[T, U any](items []T, fn func(T) U) []U {
    result := make([]U, len(items))
    for i, v := range items {
        result[i] = fn(v)
    }
    return result
}

func Filter[T any](items []T, pred func(T) bool) []T {
    var result []T
    for _, v := range items {
        if pred(v) {
            result = append(result, v)
        }
    }
    return result
}

// Usage
names := []string{"Go", "C#", "Rust", "Python"}
short := Filter(names, func(s string) bool { return len(s) <= 3 })
upper := Map(short, strings.ToUpper)
// upper: ["GO", "C#"]
```

WARNING: Go community guideline: Don't use generics just because you can. If a concrete type or interface works, prefer it. Generics shine for data structures, collection utilities, and type-safe containers.

Lesson 20: Struct Tags & Reflection

Struct tags are Go's metadata system — like C# attributes ([JsonProperty], [Required], [Column]). Reflection (the reflect package) reads these at runtime. Together they power JSON serialization, ORMs, validation, and more.

20.1 — Struct Tags Syntax

```
type User struct {
    ID          int        `json:"id" db:"user_id"`
    Name        string      `json:"name" validate:"required,min=2"`
    Email       string      `json:"email,omitempty"`
    Password    string      `json:"-"` // excluded from JSON
    CreatedAt   time.Time `json:"created_at" db:"created_at"`
}

// Tag format: `key:"value" key2:"value2"`
// Common tag keys: json, xml, yaml, db, validate, gorm
```

20.2 — C# Attributes vs Go Tags

C# / .NET

```
// C# - Attributes
public class User {
    [JsonPropertyName("id")]
    public int Id { get; set; }

    [JsonPropertyName("name")]
    [Required]
    public string Name { get; set; }

    [JsonIgnore]
    public string Password { get; set; }
}
```

Go

```
// Go - Struct Tags
type User struct {
    ID          int        `json:"id"`
    Name        string      `json:"name"`
    Password    string      `json:"-"`
}

// validate tag added separately:
// Name string `json:"name"
// validate:"required"
```

20.3 — JSON Tag Options (Go 1.24+: omitzero)

```
// json tag options:
// "name"          - custom JSON key name
// ",omitempty"    - omit if zero value
// ",omitzero"     - omit if zero (Go 1.24+, more precise)
// "-"            - always exclude
// ",string"       - encode number as JSON string

type Config struct {
    Host    string `json:"host"`
    Port    int    `json:"port,omitempty"`
    Timeout int    `json:"timeout,omitzero" // Go 1.24+`
    Debug   bool   `json:"debug,string"    // "true"/"false"`
}
```

20.4 — Reading Tags with Reflection

```
import "reflect"

func PrintTags(v any) {
    t := reflect.TypeOf(v)
    for i := 0; i < t.NumField(); i++ {
        field := t.Field(i)
        jsonTag := field.Tag.Get("json")
        dbTag := field.Tag.Get("db")
        fmt.Printf("Field: %s json=%s db=%s\n",
            field.Name, jsonTag, dbTag)
    }
}

// Output for User{}:
// Field: ID json=id db=user_id
// Field: Name json=name db=
```

20.5 — Reflection Basics

```
// reflect.TypeOf – gets the TYPE metadata
// reflect.ValueOf – gets the VALUE and can modify it

func Inspect(v any) {
    val := reflect.ValueOf(v)
    typ := reflect.TypeOf(v)

    fmt.Println("Type:", typ.Name())
    fmt.Println("Kind:", val.Kind()) // struct, ptr, slice, etc.

    if val.Kind() == reflect.Struct {
        for i := 0; i < val.NumField(); i++ {
            fmt.Printf(" %s = %v\n",
                typ.Field(i).Name, val.Field(i).Interface())
        }
    }
}
```

WARNING: Reflection is slow (10-100x slower than direct access) and loses type safety. Use it for framework-level code (serialization, ORM mapping), not business logic. The reflect package is Go's equivalent of System.Reflection.

Lesson 21: Functional Patterns — First-Class Functions & Closures

Go functions are first-class values. You can assign them to variables, pass them as arguments, return them from other functions, and create closures. This enables powerful patterns that replace some OOP constructs.

21.1 — Functions as Values

```
// Assign a function to a variable
square := func(n int) int { return n * n }

// Function type declarations (like C# delegate / Func<T,TResult>)
type Predicate[T any] func(T) bool
type Transform[T, U any] func(T) U

// Using function types
var isEven Predicate[int] = func(n int) bool {
    return n%2 == 0
}
```

21.2 — Higher-Order Functions

C# / .NET

```
// C# - LINQ
var evens = numbers
    .Where(n => n % 2 == 0)
    .Select(n => n * n)
    .ToList();

// Func<int, bool> predicate
Func<int, bool> isPositive =
    n => n > 0;
```

Go

```
// Go - explicit higher-order functions
evens := Filter(numbers,
    func(n int) bool { return n%2 == 0 })
squared := Map(evens,
    func(n int) int { return n * n })

// Named function type
isPositive := func(n int) bool {
    return n > 0
}
```

21.3 — Closures

```
// Closure captures variables from enclosing scope
func Counter(start int) func() int {
    count := start
    return func() int {
        count++
        return count
    }
}

next := Counter(0)
fmt.Println(next()) // 1
fmt.Println(next()) // 2
```

```
fmt.Println(next()) // 3

// Closure with middleware pattern (very common)
func WithLogging(handler http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        handler(w, r)
        log.Printf("%s %s took %v", r.Method, r.URL.Path,
            time.Since(start))
    }
}
```

21.4 — Immediately Invoked Function (IIFE)

```
// Used with defer and goroutines
defer func() {
    if r := recover(); r != nil {
        log.Println("recovered:", r)
    }
}()

// Scoped variable initialization
result := func() int {
    // complex computation
    return 42
}()
```

TIP: Go has no LINQ, no method chaining on collections, and no extension methods. Functional patterns are explicit — you write Map, Filter, Reduce yourself (or use slices package utilities from the standard library).

Lesson 22: Builder & Functional Options Pattern

In C# you use constructors with optional parameters, the builder pattern, or dependency injection containers. Go has none of these natively. The Functional Options Pattern is the most popular Go idiom for configurable constructors with defaults.

22.1 — The Problem: Constructor Explosion

```
// Without functional options, you get this mess:
func NewServer(host string, port int) *Server
func NewServerWithTimeout(host string, port int, t time.Duration) *Server
func NewServerWithTLS(host string, port int, cert string) *Server
func NewServerWithTimeoutAndTLS(host string, port int,
    t time.Duration, cert string) *Server
// This doesn't scale.
```

22.2 — The Functional Options Pattern

```
type Server struct {
    host      string
    port      int
    timeout   time.Duration
    maxConns  int
    tlsCertPath string
    logger    *slog.Logger
}

// Option is a function that configures Server
type Option func(*Server)

// Option constructors
func WithTimeout(d time.Duration) Option {
    return func(s *Server) { s.timeout = d }
}

func WithMaxConns(n int) Option {
    return func(s *Server) { s.maxConns = n }
}

func WithTLS(certPath string) Option {
    return func(s *Server) { s.tlsCertPath = certPath }
}

func WithLogger(l *slog.Logger) Option {
    return func(s *Server) { s.logger = l }
}

// Constructor with sensible defaults + variadic options
func NewServer(host string, port int, opts ...Option) *Server {
    s := &Server{
        host:    host,
```

```

        port:    port,
        timeout: 30 * time.Second, // default
        maxConns: 100,             // default
        logger:  slog.Default(),   // default
    }
    for _, opt := range opts {
        opt(s)
    }
    return s
}

// Usage – clean, self-documenting, extensible
srv := NewServer("0.0.0.0", 8080,
    WithTimeout(60 * time.Second),
    WithMaxConns(500),
    WithTLS("/certs/server.pem"),
)

```

22.3 — C# Builder vs Go Functional Options

C# / .NET

```

// C# – Builder Pattern
var server = new ServerBuilder()
    .WithHost("0.0.0.0")
    .WithPort(8080)
    .WithTimeout(TimeSpan
        .FromSeconds(60))
    .WithMaxConnections(500)
    .Build();

// Or: new Server(
//     host: "0.0.0.0",
//     port: 8080,
//     timeout: TimeSpan...(60)
// );

```

Go

```

// Go – Functional Options
srv := NewServer("0.0.0.0", 8080,
    WithTimeout(60 * time.Second),
    WithMaxConns(500),
)

// Required params are positional.
// Optional params are functional options.
// No builder struct needed.
// Adding new options never breaks
// existing callers.

// Used in gRPC, Zap, Temporal...

```

22.4 — Config Struct Alternative

```

// Simpler alternative for many required params
type ServerConfig struct {
    Host      string    `json:"host"`
    Port      int       `json:"port"`
    Timeout   time.Duration `json:"timeout"`
    MaxConns  int       `json:"max_conns"`
}

func NewServer(cfg ServerConfig) *Server {
    if cfg.Timeout == 0 {
        cfg.Timeout = 30 * time.Second
    }
    if cfg.MaxConns == 0 {
        cfg.MaxConns = 100
    }
}

```



```
    }  
    return &Server{/* ... */}  
}  
  
// Easy to load from JSON/YAML/env  
srv := NewServer(ServerConfig{  
    Host:      "0.0.0.0",  
    Port:      8080,  
    MaxConns: 500,  
})
```

TIP: Use Functional Options for libraries / public APIs where extensibility matters. Use Config Structs for application-level configuration that loads from files/env. Both are idiomatic.

Lesson 23: Dependency Injection — Without a Framework

Coming from .NET, you are used to `IServiceCollection`, `AddScoped<T>`, `AddSingleton<T>`, and a full DI container. Go does not use DI containers. Dependencies are wired explicitly in `main()` or a setup function. This is deliberate — the Go community values explicitness over magic.

23.1 — Manual Wiring in `main()`

```
func main() {
    // 1. Create infrastructure dependencies
    db, err := sql.Open("postgres", os.Getenv("DB_URL"))
    if err != nil { log.Fatal(err) }
    defer db.Close()

    logger := slog.New(slog.NewJSONHandler(os.Stdout, nil))

    // 2. Wire up repositories (accept interfaces, return structs)
    userRepo := postgres.NewUserRepository(db)
    orderRepo := postgres.NewOrderRepository(db)

    // 3. Wire up services with dependencies
    userSvc := service.NewUserService(userRepo, logger)
    orderSvc := service.NewOrderService(orderRepo, userRepo, logger)

    // 4. Wire up HTTP handlers
    userHandler := handler.NewUserHandler(userSvc)
    orderHandler := handler.NewOrderHandler(orderSvc)

    // 5. Setup router
    mux := http.NewServeMux()
    mux.HandleFunc("GET /users/{id}", userHandler.Get)
    mux.HandleFunc("POST /orders", orderHandler.Create)

    log.Fatal(http.ListenAndServe(":8080", mux))
}
```

23.2 — C# DI Container vs Go Manual Wiring

C# / .NET

```
// C# — DI Container
var builder = WebApplication
    .CreateBuilder(args);

builder.Services
    .AddScoped<IUserRepo, PgUserRepo>()
    .AddScoped<IUserService,
UserService>()
    .AddSingleton<ILogger, SlogLogger>();
```

Go

```
// Go — Explicit wiring
func main() {
    db := mustOpenDB()
    logger := slog.Default()

    repo := postgres.NewUserRepo(db)
    svc := service.NewUserSvc(repo,
logger)
    hdl := handler.NewUserHandler(svc)
```

<pre>var app = builder.Build(); // Runtime resolution, reflection-based</pre>	<pre>// Compile-time safe, no reflection // Missing dependency = compile error }</pre>
---	--

23.3 — Interface-Based DI

```
// Define interface where it is USED (consumer side)
// Not where it is implemented (producer side)

// In package service/
type UserRepository interface {
    FindByID(ctx context.Context, id string) (*User, error)
    Save(ctx context.Context, u *User) error
}

type UserService struct {
    repo UserRepository // interface, not concrete type
    logger *slog.Logger
}

func NewUserService(repo UserRepository, l *slog.Logger) *UserService {
    return &UserService{repo: repo, logger: l}
}

// In tests – inject a mock
type mockUserRepo struct {}

func (m *mockUserRepo) FindByID(ctx context.Context, id string) (*User, error) {
    return &User{ID: id, Name: "Test"}, nil
}

func (m *mockUserRepo) Save(ctx context.Context, u *User) error {
    return nil
}

func TestUserService(t *testing.T) {
    svc := NewUserService(&mockUserRepo{}, slog.Default())
    user, err := svc.GetUser(context.Background(), "123")
    // assert...
}
```

23.4 — When Projects Get Large: Wire & Fx

For very large Go projects (50+ services), two DI tools exist. Google Wire generates wiring code at compile time (no reflection). Uber Fx uses runtime injection. Most Go projects never need either — manual wiring in `main()` works for 90% of codebases.

Tool	Approach
------	----------

Manual (idiomatic)	Explicit wiring in <code>main()</code> . Compile-time safe. No magic.
Google Wire	Code generation. Write provider functions, Wire generates the wiring.
Uber Fx	Runtime DI container. Reflection-based. Best for very large services.

WARNING: Go devs strongly prefer compile-time safety over runtime magic. If you find yourself reaching for a DI container, reconsider your architecture first. Simpler dependency graphs = simpler wiring.

Lesson 24: Repository & Service Layer Patterns — Go vs C#

C# applications typically follow Clean Architecture or Onion Architecture with `IRepository<T>`, `IUnitOfWork`, and layered abstractions. Go takes a pragmatic approach — same concepts, dramatically less ceremony.

24.1 — Typical Go Project Layout

```
myapp/
├── cmd/
│   └── server/
│       └── main.go           // entry point, wiring
├── internal/
│   ├── user/
│   │   ├── handler.go       // HTTP handlers
│   │   ├── service.go       // business logic
│   │   ├── repository.go    // interface (defined HERE)
│   │   └── model.go         // domain model
│   ├── order/
│   │   ├── handler.go
│   │   ├── service.go
│   │   ├── repository.go
│   │   └── model.go
│   └── platform/
│       └── postgres/        // implementations
│           ├── user_repo.go
│           └── order_repo.go
├── go.mod
└── go.sum
```

24.2 — Repository Pattern in Go

```
// internal/user/repository.go – interface defined by consumer
package user

type Repository interface {
    FindByID(ctx context.Context, id string) (*User, error)
    FindByEmail(ctx context.Context, email string) (*User, error)
    Create(ctx context.Context, u *User) error
    Update(ctx context.Context, u *User) error
}

// internal/platform/postgres/user_repo.go – implementation
package postgres

type UserRepository struct {
    db *sql.DB
}

func NewUserRepository(db *sql.DB) *UserRepository {
```

```
    return &UserRepository{db: db}
}

func (r *UserRepository) FindByID(ctx context.Context, id string) (*user.User,
error) {
    row := r.db.QueryRowContext(ctx,
        "SELECT id, name, email FROM users WHERE id = $1", id)
    u := &user.User{}
    err := row.Scan(&u.ID, &u.Name, &u.Email)
    if errors.Is(err, sql.ErrNoRows) {
        return nil, user.ErrNotFound
    }
    return u, err
}
```

24.3 — Service Layer

```
// internal/user/service.go
package user

type Service struct {
    repo    Repository
    logger  *slog.Logger
}

func NewService(repo Repository, logger *slog.Logger) *Service {
    return &Service{repo: repo, logger: logger}
}

func (s *Service) GetUser(ctx context.Context, id string) (*User, error) {
    s.logger.Info("fetching user", "id", id)
    u, err := s.repo.FindByID(ctx, id)
    if err != nil {
        return nil, fmt.Errorf("get user %s: %w", id, err)
    }
    return u, nil
}

func (s *Service) Register(ctx context.Context, req RegisterRequest) (*User,
error) {
    existing, _ := s.repo.FindByEmail(ctx, req.Email)
    if existing != nil {
        return nil, ErrEmailTaken
    }
    u := &User{
        ID:    uuid.NewString(),
        Name:  req.Name,
        Email: req.Email,
    }
    if err := s.repo.Create(ctx, u); err != nil {
        return nil, fmt.Errorf("register user: %w", err)
    }
    return u, nil
}
```

}

24.4 — Key Differences from C#

C# / .NET Pattern	Go Equivalent
IRepository<T> generic base	Specific interface per domain (no generic repo)
IUnitOfWork + transactions	Pass *sql.Tx or use a closure pattern
AutoMapper DTO mapping	Manual mapping (or small helper functions)
MediatR / CQRS handlers	Direct method calls on service (less indirection)
FluentValidation	Manual validation or go-playground/validator
EF Core DbContext	database/sql + sqlc or sqlx (thin query layer)
[ApiController] attributes	http.HandlerFunc closures + manual routing

24.5 — The 'internal' Directory Convention

Go enforces the internal/ directory at the compiler level. Packages under internal/ cannot be imported by code outside the parent module. This replaces C#'s internal access modifier at the package level and prevents external consumers from depending on your implementation details.

```
// myapp/internal/user/service.go
// Can be imported by myapp/cmd/server/main.go      (same module)
// CANNOT be imported by github.com/other/project   (different module)
```

24.6 — Phase 2 Summary: Type System Mindset Shifts

C# / .NET Concept	Go Equivalent
Large interfaces (10+ methods)	Small interfaces (1-3 methods) + composition
object / dynamic	any (interface{}) – use sparingly with generics
is / as pattern matching	type assertion v.(T) / type switch
Generics with variance (in/out)	Generics with constraints (simpler, no variance)
[Attributes]	Struct tags `json:"name"``

<code>System.Reflection</code>	reflect package (slower, avoid in hot paths)
<code>LINQ / Func<T, TResult></code>	First-class functions + explicit Map/Filter
Builder pattern	Functional Options pattern
DI Container (Autofac, MSDI)	Manual wiring in main() (no container)
Clean Architecture layers	Package-per-feature + internal/convention

TIP: The single most important Go design principle from Phase 2: 'Accept interfaces, return structs.' Define interfaces where they are consumed (not where they are implemented), keep them small, and wire everything explicitly.